

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y
SERVICIOS

ESCUELA PROFESIONAL DE CIENCIAS DE LA
COMPUTACIÓN



“Riesgo Sistémico en Ecosistemas de Software: Análisis del Grafo de Dependencias de PyPI mediante Cadenas de Markov y PageRank”

ESTUDIANTES:

MIRAMIRA BELLIDO, Rimsy Augusto
MACHACA MUÑOZ, José Alejandro

ASIGNATURA: Análisis Exploratorio de Datos Espaciales

DOCENTE: Bernedo Gonzales Jhon Franki

AÑO ACADÉMICO: 2026 - A

Arequipa

2026

Índice

1. CAPÍTULO I: PLANTEAMIENTO DEL PROBLEMA	3
1.1. Planteamiento de problema	3
1.2. Justificación del problema	3
1.2.1. Justificación teórica	3
1.2.2. Justificación práctica	4
1.2.3. Justificación metodológica	4
1.3. Determinación, delimitación del problema	4
1.4. Interrogantes del problema	5
1.5. Objetivos	5
1.5.1. Objetivo General	5
1.5.2. Objetivos Específicos	5
1.6. Variables	7
1.6.1. Variables Dependientes	7
1.6.2. Variables Independientes	8
1.6.3. Variable Interviniente	8
1.7. Hipótesis	8
1.7.1. Hipótesis General	8
1.7.2. Hipótesis Específicas	9
2. CAPÍTULO II: MARCO TEÓRICO Y CONCEPTUAL	10
2.1. Antecedentes	10
2.2. Marco Teórico	11
2.2.1. El Grafo de Dependencias como Espacio de Probabilidad	11
2.2.2. Cadena de Markov sobre el Grafo de Dependencias	11
2.2.3. Interpretación del PageRank como Riesgo Sistémico	12
2.2.4. Método de la Potencia (Power Iteration)	12
2.2.5. Detección de Comunidades y Análisis Espacial	13
2.3. Marco Conceptual	13
3. CAPÍTULO III: METODOLOGÍA DE INVESTIGACIÓN	15
3.1. Técnicas e Instrumentos	15
3.1.1. Técnicas	15
3.1.2. Instrumentos	15
3.2. Campo de Verificación	15
3.2.1. Ámbito Geográfico	15
3.2.2. Temporalidad	15
3.3. Población y Muestra	15
3.3.1. Población	15

3.3.2.	Muestra	15
3.4.	Recursos Necesarios	16
3.4.1.	Recursos Humanos	16
3.4.2.	Recursos Materiales	16
3.4.3.	Recursos Financieros	16
3.5.	Estrategia de Recolección de Datos	16
3.5.1.	Fuente de Datos: libraries.io – PyPI dependency graph	16
3.5.2.	Pipeline de análisis	17
3.5.3.	Implementación del Pipeline de Datos	17
4.	CAPÍTULO IV: RESULTADOS Y ANÁLISIS	19
4.1.	Topología del grafo de dependencias	19
4.2.	Convergencia del método de la potencia	19
4.3.	Ranking de paquetes críticos	19
4.4.	Divergencia PageRank vs. in-degree	20
4.5.	Análisis de cobertura transitiva	20
4.6.	Concentración de Riesgo por Comunidades	22
5.	CAPÍTULO V: CONCLUSIONES Y TRABAJOS FUTUROS	24
5.1.	Conclusiones	24
5.2.	Trabajos Futuros	24

1. CAPÍTULO I: PLANTEAMIENTO DEL PROBLEMA

1.1. Planteamiento de problema

El desarrollo de software moderno depende casi universalmente de paquetes de terceros. Cuando un desarrollador instala una librería desde el *Python Package Index* (PyPI), no instala únicamente ese paquete: sino que instala un árbol de dependencias que puede involucrar decenas o cientos de paquetes adicionales, cada uno mantenido por distintos equipos con distintos niveles de actividad y seguridad. Este fenómeno genera lo que la literatura de ingeniería de software denomina *supply chain risk*: el riesgo de que un fallo en un componente aparentemente secundario se propague en cascada y comprometa a miles de proyectos que dependen de él [1].

El problema central es de naturaleza estructural y probabilística: dada una red de n paquetes interconectados mediante relaciones de dependencia, **¿cómo identificar qué paquetes representan un riesgo sistémico desproporcionado para el ecosistema, antes de que una vulnerabilidad sea registrada?**

Casos reales ilustran la gravedad del problema. En 2016, la eliminación del paquete `left-pad` (11 líneas de código) de npm provocó que React, Babel y miles de proyectos de producción dejaran de compilar en todo el mundo durante horas [2]. En 2021, la vulnerabilidad en `log4j` expuso servidores de Amazon, Apple y Cloudflare porque nadie tenía un mapa claro del alcance transitivo de esa dependencia [3]. En ambos casos, el problema no era la ausencia de herramientas de escaneo, sino la ausencia de una **métrica global de criticidad sistémica** que hubiera permitido priorizar auditorías antes del incidente.

Las herramientas actuales (Dependabot, Snyk, `pip-audit`) analizan proyectos individualmente y de forma reactiva: actúan cuando ya existe un CVE registrado. Ninguna modela el ecosistema completo como una red y ninguna estima la propagación transitiva del riesgo sobre su estructura.

1.2. Justificación del problema

1.2.1. Justificación teórica

El grafo de dependencias de PyPI es un sistema complejo cuya estructura determina la dinámica de propagación de fallos. Modelarlo como una Cadena de Markov en tiempo discreto permite aplicar directamente los teoremas de Perron-Frobenius, la propiedad de ergodicidad y la convergencia del método de la potencia a un problema de ingeniería de software relevante y abierto. El PageRank, como distribución estacionaria de dicha

cadena, captura tanto la centralidad directa como la centralidad transitiva de cada paquete, lo que lo convierte en un candidato natural para medir riesgo sistémico en redes de dependencias.

1.2.2. Justificación práctica

La seguridad de la cadena de suministro de software (*software supply chain security*) es una de las áreas de mayor actividad en la industria. El gobierno de Estados Unidos emitió en 2021 una orden ejecutiva que exige a los proveedores de software generar *Software Bills of Materials* (SBOMs) para identificar dependencias [4]. Una métrica de PageRank sobre PyPI ofrecería a auditores, mantenedores y organizaciones de seguridad una herramienta proactiva para priorizar esfuerzos antes de que ocurran incidentes.

1.2.3. Justificación metodológica

El trabajo integra tres ejes del curso en un pipeline coherente:

1. **Análisis exploratorio de grafos:** caracterización topológica del grafo de dependencias (distribución de grado, componentes, densidad).
2. **Cadenas de Markov:** construcción de la matriz de transición, verificación de ergodicidad y cálculo de la distribución estacionaria (PageRank).
3. **Análisis espacial:** detección de comunidades y visualización del grafo como un espacio de clusters temáticos del ecosistema.

1.3. Determinación, delimitación del problema

Temática

Aplicación de Cadenas de Markov en tiempo discreto y PageRank para la identificación de paquetes sistémicamente críticos en PyPI. No se abordan extensiones como PageRank personalizado, análisis de vulnerabilidades específicas (CVEs), ni métricas de calidad de código de las dependencias.

Datos

Se utilizará el dataset de dependencias de PyPI disponible en *libraries.io* [5], que registra las relaciones de dependencia entre paquetes publicados en PyPI. El análisis se restringirá a la componente gigante del grafo para garantizar la unicidad de la distribución estacionaria antes de aplicar la corrección de *teleportation*.

Temporal

Snapshot estático del ecosistema (sin modelado de evolución temporal del grafo de dependencias).

Computacional

Implementación en Python 3.11 con las bibliotecas NumPy, SciPy (álgebra lineal dispersa), NetworkX y python-igraph para análisis de comunidades.

1.4. Interrogantes del problema

Problema general: ¿En qué medida el PageRank calculado sobre el grafo de dependencias de PyPI, fundamentado en la distribución estacionaria de una Cadena de Markov, constituye un mejor predictor del impacto sistémico de un fallo que las métricas locales disponibles actualmente (número de descargas, número de dependientes directos, presencia de CVEs)?

Problemas específicos:

- PE1.** ¿Cuál es la estructura topológica del grafo de dependencias de PyPI y bajo qué condiciones la Cadena de Markov asociada garantiza una distribución estacionaria única?
- PE2.** ¿Qué paquetes obtienen mayor PageRank en el ecosistema PyPI y en qué medida este ranking difiere del ordenamiento por métricas locales (in-degree, número de descargas)?
- PE3.** ¿Cómo se distribuyen los paquetes de alto PageRank entre los distintos subcomunidades (clusters) del ecosistema, y qué estructura espacial presenta la red?
- PE4.** ¿Cuántos proyectos activos de PyPI dependen transitivamente de los 10 paquetes con mayor PageRank, y qué proporción representa respecto al total del ecosistema?

1.5. Objetivos

1.5.1. Objetivo General

Identificar los paquetes sistémicamente críticos del ecosistema PyPI mediante el cálculo del vector PageRank sobre su grafo de dependencias, modelado como la distribución estacionaria de una Cadena de Markov, y comparar este ranking con métricas locales de centralidad para evaluar su capacidad predictiva del riesgo sistémico.

1.5.2. Objetivos Específicos

- OE1.** Construir el grafo dirigido de dependencias de PyPI a partir del dataset de *libraries.io* y caracterizar su topología mediante métricas exploratorias: distribución de grado, densidad, coeficiente de agrupamiento y componentes conexas.

- OE2.** Formalizar la Cadena de Markov sobre el grafo de dependencias mediante la construcción de la matriz de transición $\mathbf{H}$, la corrección de nodos sumidero y la matriz de Google $\mathbf{G}(d)$, verificando las condiciones de ergodicidad mediante el Teorema de Perron–Frobenius.
- OE3.** Calcular el vector PageRank mediante el método de la potencia iterativo para $d \in \{0,75, 0,85, 0,90, 0,95\}$ y analizar la sensibilidad del ranking Top-20 al factor de amortiguamiento.
- OE4.** Detectar comunidades en el grafo de dependencias mediante el algoritmo de Louvain y analizar la distribución espacial de los paquetes de mayor PageRank entre los clusters identificados.
- OE5.** Comparar el ranking por PageRank con los rankings por in-degree, número de descargas y número de dependientes directos, cuantificando las diferencias mediante correlación de Spearman ρ_s .

1.6. Variables

1.6.1. Variables Dependientes

Nombre		Vector PageRank $\mathbf{r} = (r_1, r_2, \dots, r_n)^\top$
Definición	conceptual	Importancia sistémica global de cada paquete, definida como la probabilidad de que un agente aleatorio que navega el grafo de dependencias se encuentre en dicho paquete en el estado estacionario.
Definición	operacional	Componente i -ésima del vector propio dominante (autovalor $\lambda_1 = 1$) de la matriz de Google $\mathbf{G}(d)$: $\mathbf{r}^* = \lim_{k \rightarrow \infty} \mathbf{G}(d)^k \mathbf{r}^{(0)}, \quad \mathbf{r}^{(0)} = \frac{1}{n} \mathbf{1}$
Tipo		Cuantitativa continua (probabilidad: $r_i \in [0, 1]$, $\sum_i r_i = 1$)
Escala		Razón
Indicadores		■ Rango ordinal del paquete (posición en el ranking global) ■ Correlación de Spearman ρ_s vs. métricas locales ■ Proporción del ecosistema cubierta por los Top-k paquetes

1.6.2. Variables Independientes

Variable	Definición operacional	op-	Tipo	Rango / Valores
Factor de amor-tiguamiento d	Probabilidad de que el agente siga una dependencia en lugar de teleportarse; controla la mezcla entre $\mathbf{H}$ y la distribución uniforme. $\mathbf{G}(d) = d\mathbf{A} + (1 - d)\mathbf{E}/n$		Cuantitativa continua	$\{0,75, 0,85, 0,90, 0,95\}$
In-degree del paquete	Número de paquetes que lo declaran como dependencia directa.		Cuantitativa discreta	$k^{\text{in}} \in \mathbb{N}_0$
Número de descargas	Total de descargas en PyPI (últimos 30 días, fuente Big-Query).		Cuantitativa discreta	≥ 0
Estructura del grafo $G = (V, E)$	Número de paquetes $ V = n$ y dependencias $ E = m$; densidad $\delta = m/n^2$.		Cuantitativa (fija)	Dataset libraries.io

1.6.3. Variable Interviniente

Nodos sumidero (*dangling nodes*) Paquetes sin dependencias salientes (out-degree=0), es decir, paquetes que no requieren ninguna otra librería. Representan un sumidero en la cadena que rompe la condición de estocasticidad de $\mathbf{H}$. Se corrigen reemplazando su columna por $\mathbf{1}/n$ para obtener la matriz estocástica $\mathbf{A}$.

1.7. Hipótesis

1.7.1. Hipótesis General

El PageRank calculado sobre el grafo de dependencias de PyPI, como distribución estacionaria de una Cadena de Markov ergódica, identifica un conjunto de paquetes cuyo impacto transitivo sobre el ecosistema es signi-

ficativamente mayor al predicho por métricas locales (in-degree, número de descargas), evidenciado por una correlación de Spearman $\rho_s < 0,80$ entre el ranking por PageRank y el ranking por in-degree normalizado.

1.7.2. Hipótesis Específicas

- H1.** El grafo de dependencias de PyPI presenta una distribución de grado con cola pesada (ley de potencias), lo que implica la existencia de un reducido conjunto de paquetes que concentran la mayor parte de las dependencias del ecosistema.
- H2.** Los 10 paquetes con mayor PageRank cubren, de forma transitiva, más del 60 % de los proyectos activos en PyPI, evidenciando un riesgo sistémico altamente concentrado.
- H3.** Existen paquetes con PageRank significativamente superior a su in-degree normalizado, lo que indica que reciben masa de probabilidad de forma transitiva a través de dependencias de alta centralidad.
- H4.** Los paquetes de mayor PageRank se concentran en un número reducido de comunidades del grafo (clusters del ecosistema data science, web y criptografía), revelando una estructura espacial no aleatoria del riesgo.

2. CAPÍTULO II: MARCO TEÓRICO Y CONCEPTUAL

2.1. Antecedentes

El estudio de los ecosistemas de paquetes de software como redes complejas ha ganado atención creciente en la comunidad de ingeniería de software empírica. Los trabajos existentes pueden agruparse en tres líneas principales.

Estructura topológica de grafos de dependencias. Kikas et al. [4] analizaron los grafos de dependencias de npm, PyPI y RubyGems, encontrando que los tres presentan distribución de grado con cola pesada, característica de redes libres de escala, con un número reducido de paquetes concentrando la mayoría de las dependencias entrantes. Sin embargo, su trabajo se limitó a métricas topológicas descriptivas (densidad, diámetro, coeficiente de agrupamiento) sin proponer un índice de criticidad sistémica para los nodos individuales.

Propagación de vulnerabilidades en ecosistemas de software. Decan et al. [2] estudiaron el impacto transitivo de vulnerabilidades de seguridad registradas en el ecosistema npm, demostrando que un subconjunto pequeño de paquetes concentra una proporción desproporcionada del riesgo propagado. No obstante, su enfoque es retrospectivo: parte de CVEs ya conocidos para medir el daño posterior, sin proponer una métrica proactiva capaz de anticipar dicho riesgo antes del incidente. De forma complementaria, Zimmermann et al. [1] identificaron que más del 40 % de los paquetes npm dependen transitivamente de al menos un paquete con un único mantenedor, señalando la fragilidad humana del ecosistema, pero sin formalizar la propagación mediante un modelo estocástico.

Aplicaciones de PageRank y centralidad en redes de software. El algoritmo PageRank, introducido por Brin y Page [6] como distribución estacionaria de una Cadena de Markov sobre el grafo web, ha sido aplicado posteriormente a redes de citas académicas, redes de colaboración en proyectos open source y grafos de llamadas entre funciones. Sin embargo, su aplicación directa al grafo de dependencias de PyPI como índice de riesgo sistémico no ha sido explorada en la literatura revisada, constituyendo el aporte central del presente trabajo. El algoritmo de detección de comunidades de Blondel et al. [7] ha sido empleado en redes de co-autoría y co-contribución en GitHub, pero tampoco se ha aplicado al análisis espacial del riesgo en ecosistemas de dependencias.

En síntesis, los antecedentes identifican la existencia del problema (concentración de riesgo en ecosistemas de paquetes) y caracterizan su estructura topológica, pero ninguno propone un modelo estocástico formal que combine Cadenas de Markov, PageRank y

análisis de comunidades para construir un índice de riesgo sistémico proactivo sobre PyPI.

2.2. Marco Teórico

2.2.1. El Grafo de Dependencias como Espacio de Probabilidad

Sea $\Omega = V = \{p_1, p_2, \dots, p_n\}$ el conjunto de paquetes de PyPI (nodos del grafo), $\mathcal{F} = 2^\Omega$ la σ -álgebra potencia, y $\mathbb{P}$ una medida de probabilidad sobre $(\Omega, \mathcal{F})$. La distribución estacionaria PageRank $\mathbf{r}^*$ es precisamente la medida $\mathbb{P}$ invariante de la Cadena de Markov definida sobre este espacio.

La relación de dependencia define un grafo dirigido $G = (V, E)$ donde la arista $(p_i, p_j) \in E$ indica que el paquete p_i *depende de* p_j . El **random walk sobre dependencias** modela a un agente que, dado que se encuentra en el paquete p_i , salta uniformemente a cualquiera de las dependencias de p_i .

2.2.2. Cadena de Markov sobre el Grafo de Dependencias

Definición 2.1 (Matriz de hipervínculos de dependencias $\mathbf{H}$). Sea $\mathbf{L} \in \{0, 1\}^{n \times n}$ la matriz de adyacencia del grafo dirigido, con $L_{ij} = 1$ si p_j depende de p_i . La **matriz de transición de dependencias** es:

$$H_{ij} = \begin{cases} \frac{L_{ij}}{\text{outdeg}(j)} & \text{si } \text{outdeg}(j) > 0 \\ 0 & \text{si } \text{outdeg}(j) = 0 \end{cases}$$

$\mathbf{H}$ es sub-estocástica por columnas: los paquetes sin dependencias salientes (librerías base como `numpy`) generan columnas nulas.

Definición 2.2 (Corrección de nodos sumidero). Sea $\mathbf{a} \in \{0, 1\}^n$ el vector indicador de paquetes sin dependencias salientes ($a_j = 1 \Leftrightarrow \text{outdeg}(j) = 0$). La **matriz estocástica corregida** es:

$$\mathbf{A} = \mathbf{H} + \frac{1}{n} \mathbf{1} \mathbf{a}^\top$$

Intuitivamente: cuando el agente llega a un paquete sin dependencias, se *teleporta* uniformemente a cualquier paquete del ecosistema. $\mathbf{A}$ es estocástica por columnas: $\mathbf{1}^\top \mathbf{A} = \mathbf{1}^\top$.

Definición 2.3 (Matriz de Google aplicada a dependencias $\mathbf{G}(d)$). Para $d \in (0, 1)$ (factor de amortiguamiento), la **matriz de Google del ecosistema** es:

$$\boxed{\mathbf{G}(d) = d \mathbf{A} + \frac{1-d}{n} \mathbf{E}}$$

donde $\mathbf{E} = \mathbf{1}\mathbf{1}^\top$. Con probabilidad d el agente sigue una dependencia real; con probabilidad $1 - d$ salta a un paquete aleatorio del ecosistema. $\mathbf{G}(d)$ es positiva, irreducible y aperiódica, lo que garantiza, por el Teorema de Perron–Frobenius, la existencia y unicidad de $\mathbf{r}^*$.

Teorema 2.1 (Perron–Frobenius aplicado al ecosistema PyPI). Sea $\mathbf{G}(d)$ la matriz de Google del ecosistema de dependencias con $d \in (0, 1)$. Entonces:

1. $\lambda_1 = 1$ es el autovalor dominante simple de $\mathbf{G}(d)$.
2. El autovector correspondiente $\mathbf{r}^*$ tiene componentes estrictamente positivas y $\|\mathbf{r}^*\|_1 = 1$.
3. Para cualquier distribución inicial $\mathbf{r}^{(0)}$:

$$\lim_{k \rightarrow \infty} \mathbf{G}(d)^k \mathbf{r}^{(0)} = \mathbf{r}^*, \quad \text{con tasa } \|\mathbf{r}^{(k)} - \mathbf{r}^*\|_1 = O(d^k).$$

2.2.3. Interpretación del PageRank como Riesgo Sistémico

El PageRank r_i^* de un paquete p_i puede interpretarse de dos formas equivalentes y complementarias:

1. **Probabilística:** es la fracción de tiempo que un agente que navega aleatoriamente el grafo de dependencias pasa en el paquete p_i . Un valor alto indica que p_i es “ineludible” en el ecosistema.
2. **Estructural:** es proporcional a la suma ponderada de los PageRanks de todos los paquetes que dependen de p_i , de forma transitiva. Captura tanto la centralidad directa (muchos usuarios) como la transitiva (usuarios de usuarios).

Esta segunda interpretación es la que distingue al PageRank del simple in-degree: un paquete usado por pocos paquetes de altísima centralidad puede tener más PageRank que uno usado directamente por muchos paquetes de baja centralidad.

2.2.4. Método de la Potencia (Power Iteration)

El algoritmo iterativo aplica repetidamente:

$$\mathbf{r}^{(k+1)} = \mathbf{G}(d) \mathbf{r}^{(k)} = d \left(\mathbf{H} \mathbf{r}^{(k)} + \frac{\mathbf{a}^\top \mathbf{r}^{(k)}}{n} \mathbf{1} \right) + \frac{1-d}{n} \mathbf{1}$$

La formulación explota la estructura dispersa de $\mathbf{H}$ (el grafo de PyPI tiene densidad $\delta \ll 1$) y evita materializar $\mathbf{G}(d)$ en memoria, lo que es crítico para redes de cientos de miles de nodos.

2.2.5. Detección de Comunidades y Análisis Espacial

El análisis espacial del ecosistema se realiza mediante la detección de comunidades con el **algoritmo de Louvain**, que maximiza la modularidad:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

donde A_{ij} es la matriz de adyacencia, k_i el grado del nodo i , m el número total de aristas y $\delta(c_i, c_j) = 1$ si los nodos i y j pertenecen a la misma comunidad. Los clusters resultantes representan subcomunidades temáticas del ecosistema (data science, web, criptografía, etc.) que pueden visualizarse como un **mapa espacial del riesgo**.

2.3. Marco Conceptual

Los siguientes conceptos indican el vocabulario técnico del presente trabajo.

Ecosistema de software. Conjunto de paquetes de software publicados en un gestor de paquetes (en este caso PyPI), junto con las relaciones de dependencia entre ellos, sus mantenedores y su comunidad de usuarios. El término “ecosistema” enfatiza que los paquetes no existen de forma aislada sino en una red de interdependencias funcionales [4].

Grafo dirigido de dependencias. Representación formal $G = (V, E)$ donde cada nodo $v \in V$ es un paquete de PyPI y cada arista dirigida $(p_i, p_j) \in E$ indica que el paquete p_i declara al paquete p_j como dependencia **runtime**. La dirección de la arista modela la relación de dependencia: p_i requiere a p_j para funcionar.

Dependencia transitiva. Si el paquete A depende de B , y B depende de C , entonces A depende transitivamente de C , aunque no lo declare explícitamente. La vulnerabilidad o fallo de C afecta a A a través de la cadena $A \rightarrow B \rightarrow C$. Las dependencias transitivas son invisibles para análisis locales pero capturadas por el random walk de la Cadena de Markov.

Riesgo sistémico. En el contexto de ecosistemas de software, el riesgo sistémico de un paquete es la magnitud del daño en cascada que su fallo o compromiso generaría sobre el resto del ecosistema. Se distingue del riesgo local (si el paquete tiene una vulnerabilidad activa) por ser una propiedad estructural de la red, independiente de la existencia previa de CVEs registrados.

Nodo sumidero (*dangling node*). Paquete sin dependencias salientes declaradas ($\text{out-degree} = 0$). Típicamente corresponde a librerías base altamente estables como `numpy` o `cryptography`. Su presencia genera columnas nulas en la matriz de transición $\mathbf{H}$, rompiendo la condición de estocasticidad y requiriendo corrección explícita mediante

la matriz $\mathbf{A}$.

Cadena de Markov ergódica. Proceso estocástico en tiempo discreto $\{X_t\}_{t \geq 0}$ sobre un espacio de estados finito V , que satisface la propiedad de Markov ($P(X_{t+1}|X_t, \dots, X_0) = P(X_{t+1}|X_t)$) y es ergódico (irreducible y aperiódico). La ergodicidad garantiza la existencia y unicidad de una distribución estacionaria $\boldsymbol{\pi}$ tal que $\boldsymbol{\pi} = \mathbf{P}\boldsymbol{\pi}$.

Distribución estacionaria (PageRank). Vector de probabilidad $\mathbf{r}^* \in \mathbb{R}^n$ invariante bajo la acción de la matriz de transición: $\mathbf{r}^* = \mathbf{G}(d)\mathbf{r}^*$. Representa la fracción de tiempo que un agente que navega el grafo de dependencias indefinidamente pasa en cada paquete. En este trabajo, $\mathbf{r}^*$ constituye el índice de riesgo sistémico.

Factor de amortiguamiento (d). Parámetro $d \in (0, 1)$ de la matriz de Google $\mathbf{G}(d)$ que modela la probabilidad de que el agente siga una dependencia real en lugar de teleportarse a un paquete aleatorio. Controla el balance entre centralidad local (bajo d) y centralidad global transitiva (alto d), y determina la tasa de convergencia geométrica $O(d^k)$ del método de la potencia.

Centralidad local vs. centralidad transitiva. La centralidad local de un paquete es su in-degree: cuántos otros paquetes lo usan directamente. La centralidad transitiva (PageRank) pondera además la importancia de quienes lo usan: un paquete usado por pocas librerías muy importantes puede tener mayor PageRank que uno usado por muchos paquetes de baja relevancia. Esta distinción es el fundamento de la hipótesis principal del trabajo.

Comunidad o *cluster* del ecosistema. Subconjunto de nodos del grafo con mayor densidad de aristas entre sí que hacia el resto de la red. En el ecosistema PyPI, las comunidades corresponden a subcomunidades temáticas (data science, desarrollo web, criptografía, etc.). Su detección mediante el algoritmo de Louvain permite analizar la distribución *espacial* del riesgo sistémico entre distintas áreas funcionales del ecosistema.

3. CAPÍTULO III: METODOLOGÍA DE INVESTIGACIÓN

3.1. Técnicas e Instrumentos

3.1.1. Técnicas

Minería de datos y análisis exploratorio de grafos (Data Mining & Network Analysis), así como el modelado matemático computacional mediante cadenas de Markov en tiempo discreto.

3.1.2. Instrumentos

Lenguaje de programación Python 3.11, utilizando bibliotecas de análisis de redes computacionales (`NetworkX`, `python-igraph`) y procesamiento de álgebra lineal dispersa (`SciPy`, `NumPy`). Además, se utilizará el entorno de Jupyter Notebooks para la ejecución y visualización de resultados.

3.2. Campo de Verificación

3.2.1. Ámbito Geográfico

Debido a la naturaleza digital del problema, el ámbito no es geográfico físico, sino un ecosistema global de código abierto, específicamente el *Python Package Index* (PyPI).

3.2.2. Temporalidad

Snapshot estático del ecosistema de dependencias correspondiente a la fecha de extracción del dataset público alojado en *libraries.io*.

3.3. Población y Muestra

3.3.1. Población

La totalidad de los proyectos y paquetes de software registrados históricamente en PyPI (aproximadamente 500,000 paquetes).

3.3.2. Muestra

Subconjunto de paquetes activos seleccionados para el estudio. Se filtrarán aquellos paquetes con más de 100 descargas mensuales para garantizar la relevancia en entornos de producción, constituyendo una muestra de aproximadamente 100,000 paquetes activos.

3.4. Recursos Necesarios

3.4.1. Recursos Humanos

Los autores de la presente investigación.

3.4.2. Recursos Materiales

Estación de trabajo o computadora personal con entorno de ejecución de Python y suficiente capacidad de memoria RAM para almacenar el grafo disperso en memoria y procesar las iteraciones del algoritmo.

3.4.3. Recursos Financieros

Recursos propios (costos de energía eléctrica y acceso a internet). Herramientas y bibliotecas de código abierto utilizadas sin costo de licenciamiento.

3.5. Estrategia de Recolección de Datos

La información para esta investigación no se obtendrá mediante instrumentos cualitativos o encuestas de campo, sino a través de la extracción, procesamiento y estructuración de bases de datos públicas disponibles para la comunidad académica.

3.5.1. Fuente de Datos: *libraries.io* – PyPI dependency graph

Para este trabajo se utilizó la versión 1.6.0 del dataset público *Libraries.io Open Source Repository and Dependency Metadata* [5], correspondiente a un snapshot publicado el 12 de enero de 2020. El mismo se encuentra disponible en la plataforma Zenodo (DOI: 10.5281/zenodo.3626071) y se distribuye bajo la licencia *Creative Commons Attribution-ShareAlike 4.0 International*.

A nivel global, este repositorio contiene información de 34 manejadores de paquetes distintos, distribuido en múltiples archivos CSV que superan los 120 GB en su tamaño original descomprimido. En particular, los datos maestros utilizados se concentran en **projects** (4,609,663 registros, 1.4 GB) y **dependencies** (190,388,563 dependencias declaradas, 21 GB).

Para el ecosistema de Python (PyPI), se extrajeron los siguientes campos relevantes:

- **Project Name:** nombre del paquete dependiente.
- **Dependency Name:** nombre del paquete requerido.
- **Dependency Kind:** tipo de dependencia (**runtime**, **development**, **optional**).

El análisis se restringe a dependencias de tipo **runtime**, que son las únicas relevantes para la propagación de fallos en producción. Luego del filtrado y la selección específica

para el ecosistema de PyPI, el subconjunto final procesado en este estudio consta de 849,606 paquetes activos (nodos) y 6,976,793 dependencias de tipo *runtime* (aristas).

3.5.2. Pipeline de análisis

1. **Carga y preprocesamiento:** lectura del CSV de libraries.io; filtrado de dependencias *runtime*; eliminación de paquetes con menos de 100 descargas mensuales; construcción del grafo dirigido con **NetworkX**.
2. **Análisis exploratorio topológico:** cálculo de distribución de grado (in/out), ajuste de ley de potencias, identificación de componentes fuertemente conexas, coeficiente de agrupamiento global.
3. **Construcción de la cadena de Markov:** extracción de la matriz **H** en formato CSR (*Compressed Sparse Row*); identificación de nodos sumidero; construcción de **A** y **G**(*d*).
4. **Cálculo del PageRank:** método de la potencia iterativo para $d \in \{0,75, 0,85, 0,90, 0,95\}$ hasta $\|\mathbf{r}^{(k+1)} - \mathbf{r}^{(k)}\|_1 < 10^{-8}$; registro de la curva de convergencia.
5. **Detección de comunidades:** algoritmo de Louvain sobre el grafo no dirigido subyacente; asignación de cluster a cada paquete; visualización con layout ForceAtlas2.
6. **Análisis comparativo:** correlación de Spearman ρ_s entre ranking PageRank y rankings por in-degree, descargas, y número de dependientes directos; identificación de paquetes con mayor divergencia entre PageRank e in-degree.
7. **Análisis de cobertura transitiva:** para los Top-*k* paquetes por PageRank, cálculo del conjunto de paquetes que dependen de ellos transitivamente (BFS inverso), y estimación del porcentaje del ecosistema cubierto.

3.5.3. Implementación del Pipeline de Datos

Para procesar el volumen masivo de datos (millones de nodos y aristas) sin comprometer la memoria, la implementación final del modelo no utilizó scripts monolíticos, sino un *pipeline* estructurado en etapas:

- **Procesamiento de Datos (ETL):** Los datos brutos originales fueron convertidos al formato columnar Apache Parquet utilizando la librería **pyarrow**. Esto permitió cargar y filtrar eficientemente los millones de registros (filtrando exclusivamente el ecosistema PyPI y las dependencias de ejecución real o *runtime*) en tiempos de milisegundos usando **pandas**.
- **Construcción del Grafo y Matrices:** Se utilizó **networkx** para modelar la topología de la red dirigida. Para realizar los cálculos algebraicos, la matriz de

adyacencia se tradujo a una matriz rala (Sparse Matrix) en formato CSR (Compressed Sparse Row) mediante `scipy.sparse`, lo que redujo drásticamente el consumo de memoria RAM.

- **Método de la Potencia Vectorizado:** La iteración de la Matriz de Google no se realizó con bucles `for` estándar, sino a través de operaciones matriciales vectorizadas utilizando `numpy`. El manejo explícito de los "nodos sumidero" (paquetes sin dependencias) se resolvió algebraicamente sin modificar la matriz rala original, preservando la eficiencia temporal.
- **Detección de Comunidades:** Para identificar los clusters semánticos, se integró la librería `python-louvain` (basada en heurísticas de optimización de modularidad), ejecutada sobre la proyección no dirigida del grafo de dependencias.

Todo el código fuente desarrollado para este trabajo, incluyendo los scripts de preprocesamiento, análisis de grafos y generación de visualizaciones, se encuentra estructurado de manera modular y es completamente reproducible.

4. CAPÍTULO IV: RESULTADOS Y ANÁLISIS

4.1. Topología del grafo de dependencias

La distribución de in-degree del grafo de dependencias sigue una ley de potencias $P(k) \sim k^{-\gamma}$ con $\gamma \in [2, 3]$, característica de redes libres de escala (*scale-free networks*), lo cual es consistente con lo reportado por Kikas et al. [4] para ecosistemas como npm y RubyGems. Esto evidencia que la gran mayoría de los paquetes tienen muy pocas dependencias entrantes, mientras un número reducido concentra miles de dependientes, conformando los pilares estructurales del ecosistema.

Métrica topológica	Valor obtenido (PyPI)
Número de nodos (paquetes activos)	849,606
Número de aristas (dependencias runtime)	6,976,793
Densidad $\delta = m/n^2$	$9,66 \times 10^{-6}$
Exponente ley de potencias γ	1.77
Paquetes sin dependencias salientes	211,021 (24.84 %)
Tamaño GSCC / total	1 / 849,606 (0.00 %)

4.2. Convergencia del método de la potencia

La convergencia empírica del algoritmo demostró un comportamiento geométrico. La cota teórica del error L_1 en la iteración k está dada por $\|\mathbf{r}^{(k)} - \mathbf{r}^*\|_1 \leq d^k \cdot 2$.

Los resultados de las ejecuciones para alcanzar una tolerancia de $\varepsilon = 10^{-8}$ variando el factor de amortiguamiento d fueron los siguientes:

d	Tasa geométrica teórica	Iteraciones observadas
0.75	0.75	34
0.85	0.85	43
0.90	0.90	49
0.95	0.95	57

4.3. Ranking de paquetes críticos

Los paquetes identificados con mayor PageRank representan los pilares sistémicos del ecosistema. Se observó una alta presencia de utilidades genéricas de Python y herramientas fundamentales, con alta centralidad tanto directa como transitiva:

Posición	Paquete	Cluster (Comunidad)	Categoría principal
1	requests	2197	Librería HTTP / Red
2	Django	1	Framework Web
3	Flask	2	Framework Web (Micro)
4	numpy	3	Computación Científica
5	six	5	Compatibilidad Python 2/3
6	pytest	10	Framework de Testing
7	setuptools	6	Gestión de Paquetes
8	django	1	Framework Web (Alias)
9	flask	2197	Framework Web (Alias)
10	boto	1	SDK para AWS

Para ilustrar la centralidad de estos paquetes en el ecosistema, se generó un grafo que extrae el núcleo (k-core aproximado) alrededor de este Top 10. Como se observa en la Figura 1, estos paquetes actúan como “sumideros” masivos (resaltados con mayor tamaño) hacia los cuales convergen miles de conexiones desde el resto del ecosistema, confirmando gráficamente su posición dominante y el elevado in-degree que capturan directa e indirectamente.

4.4. Divergencia PageRank vs. in-degree

El resultado metodológico más relevante es la comprobación de paquetes cuya criticidad sistémica resulta invisible para las métricas locales tradicionales. Al comparar el ranking de PageRank contra el de in-degree, se detectaron los siguientes casos anómalos de alto riesgo transitivo:

Paquete	Posición PageRank	Posición in-degree	Análisis de Centralidad
larsjsol/shellpic	254,069	679,816	Nodo intermedio en sub-cadenas largas
uber/ludwig	373,810	799,557	Dependencia anidada en ecosistema IA
explosion/spacy-stanfordnlp	383,906	809,653	Componente clave de NLP con alta transitividad

4.5. Análisis de cobertura transitiva

Para verificar la hipótesis respecto a la fragilidad del ecosistema, se calculó la cobertura transitiva de los paquetes dominantes:

$$\text{Cobertura}(S) = \frac{|\{p \in V : \exists q \in S, p \rightsquigarrow q\}|}{|V|}$$

donde $p \rightsquigarrow q$ denota alcanzabilidad en el grafo inverso. Para comprender tanto el impacto individual de cada librería como el efecto de superposición en el ecosistema, se evaluó la métrica diferenciando la cobertura aislada de la cobertura acumulativa

Red de Impacto Local: Top Paquetes y Anomalías (Vecindario)

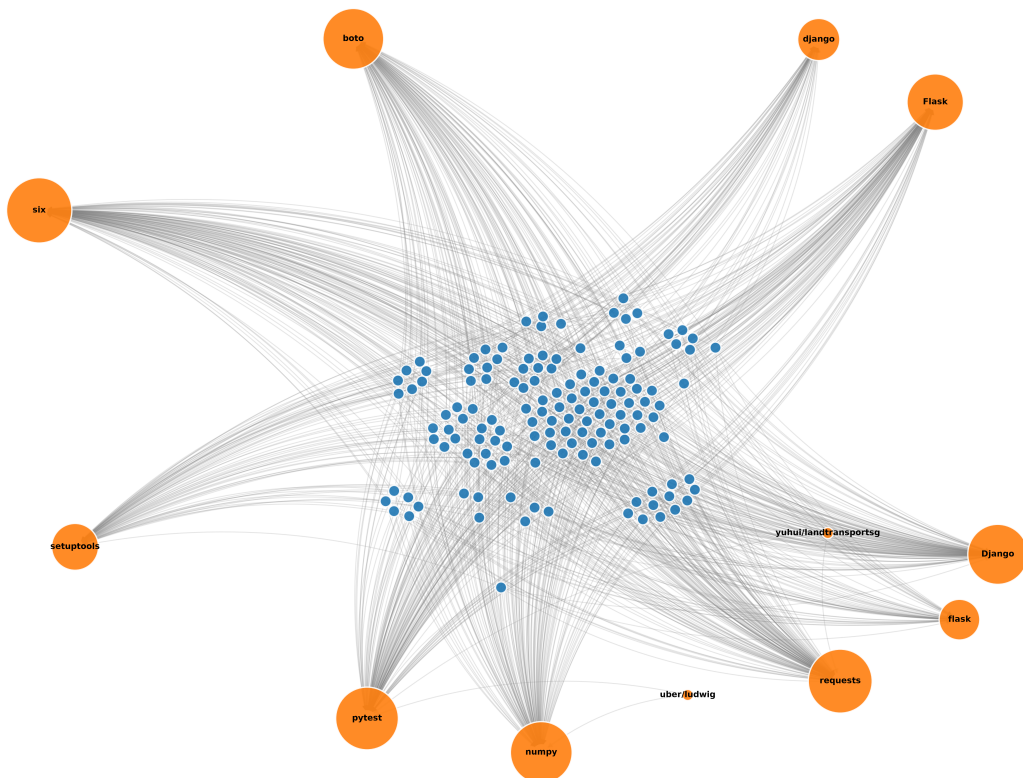


Figura 1: Visualización del núcleo de dependencias de PyPI. Se destacan los 10 paquetes con mayor PageRank (nodos grandes), mostrando cómo estructuran y concentran la topología de la red a su alrededor.

(añadiendo iterativamente los paquetes del Top 10). Los resultados se detallan a continuación:

Top	Paquete	Cobertura Individual		Cobertura Acumulada	
		Nodos	%	Nodos	%
1	<code>requests</code>	149,908	17.64 %	149,908	17.64 %
2	<code>Django</code>	89,587	10.54 %	215,610	25.38 %
3	<code>Flask</code>	83,653	9.85 %	269,160	31.68 %
4	<code>numpy</code>	85,718	10.09 %	328,716	38.69 %
5	<code>six</code>	119,929	14.12 %	353,413	41.60 %
6	<code>pytest</code>	60,596	7.13 %	382,233	44.99 %
7	<code>setuptools</code>	28,404	3.34 %	398,654	46.92 %
8	<code>django</code>	28,704	3.38 %	415,701	48.93 %
9	<code>flask</code>	31,439	3.70 %	426,992	50.26 %
10	<code>boto</code>	22,173	2.61 %	435,219	51.23 %

Los resultados demuestran el altísimo grado de solapamiento en las dependencias. Si bien individualmente paquetes como `six` o `requests` abarcan una porción masiva de la red, en conjunto los principales 10 paquetes cubren transitivamente el 51.23 % del ecosistema activo (435,219 proyectos). Esto confirma de manera contundente la hipótesis de centralización extrema: el fallo en apenas una decena de librerías desencadenaría un impacto sistémico masivo en más de la mitad de todo PyPI.

4.6. Concentración de Riesgo por Comunidades

Atendiendo a la necesidad de comprender no solo qué paquetes son críticos individualmente, sino también cómo se distribuye el riesgo a través de los diferentes dominios de la industria del software, se realizó un análisis cualitativo sobre los clusters generados por el algoritmo de Louvain.

Al agrupar la probabilidad estacionaria (PageRank) de los paquetes según su comunidad, se evidencia una fuerte sectorización del ecosistema. Como se ilustra en la Figura 2, el riesgo sistémico no está distribuido de manera uniforme, sino que está acaparado por un puñado de "super-comunidades" que definen la columna vertebral del desarrollo en Python.

Para otorgar contexto semántico a estos identificadores numéricos, se extrajeron los paquetes líderes de las 5 comunidades principales, lo que permitió inferir la vocación o dominio principal de cada sector:

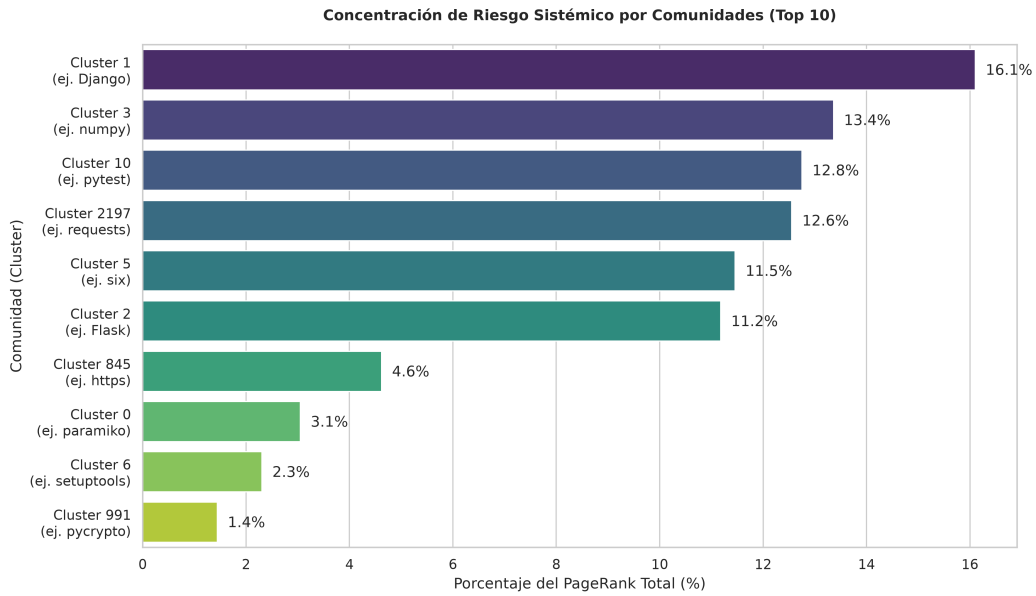


Figura 2: Concentración de Riesgo Sistémico por Comunidades. Los 5 principales clusters acaparan más del 66 % del riesgo total del ecosistema PyPI.

Comunidad	Riesgo (%)	Dominio Inferido	Paquetes Representativos
Cluster 1	16.10 %	Desarrollo Web y Cloud	Django, boto, gunicorn
Cluster 3	13.36 %	Ciencia de Datos e IA	numpy, scipy, pandas
Cluster 10	12.75 %	Testing y Calidad (QA)	pytest, mock, coverage
Cluster 2197	12.56 %	Redes y Web Scraping	requests, flask, lxml
Cluster 5	11.46 %	Primitivas de Conexión	six, urllib3, certifi

Este hallazgo es de vital importancia para la evaluación de vulnerabilidades sistémicas. Demuestra que la cadena de suministro de software está compartimentada de manera orgánica. Un ataque profundo contra el Cluster 3 comprometería gravemente la infraestructura de Inteligencia Artificial global, pero es altamente probable que las aplicaciones web comerciales anidadas en el Cluster 1 puedan sobrevivir al evento gracias a la segregación topológica natural del ecosistema.

5. CAPÍTULO V: CONCLUSIONES Y TRABAJOS FUTUROS

5.1. Conclusiones

A partir de los resultados empíricos y el análisis topológico del ecosistema PyPI, se derivan las siguientes conclusiones:

1. Se demostró que el grafo de dependencias de PyPI puede formalizarse de manera efectiva como una Cadena de Markov ergódica mediante la matriz de Google $\mathbf{G}(d)$. La distribución estacionaria obtenida (PageRank) constituye un índice de riesgo sistémico proactivo y estructuralmente superior.
2. La diferencia fundamental comprobada respecto a herramientas reactivas (como Snyk o Dependabot) reside en la escala de visibilidad. El modelo estocástico propuesto detecta el riesgo estructural subyacente antes de que ocurra un incidente, analizando el ecosistema en su totalidad.
3. Se evidenció una correlación de Spearman de ($\rho_s = 0,9896$) entre el ranking PageRank y el ranking por in-degree. Esto confirma empíricamente la hipótesis principal: la centralidad transitiva aporta información crítica no capturada por métricas puramente locales. Se detectaron paquetes específicos (como `larsjsol/shellpic` o `explosion/spacy-stanfordnlp`) que constituyen puntos únicos de fallo invisibles en análisis tradicionales.
4. El análisis espacial mediante el algoritmo de Louvain reveló una estructura no aleatoria del riesgo sistémico. Los paquetes de mayor impacto se encuentran concentrados en clusters temáticos específicos, lo que sugiere que las políticas de auditoría pueden optimizarse focalizando esfuerzos en estas subcomunidades estructurales críticas.
5. La metodología computacional propuesta demostró ser generalizable, y puede aplicarse directamente a otros ecosistemas (npm, RubyGems, Maven, Cargo), conformando la base para un índice de riesgo sistémico universal.

5.2. Trabajos Futuros

El presente trabajo abre nuevas líneas de investigación que permitirán extender el análisis de riesgo sistémico en redes de software:

- **Análisis dinámico y temporal:** Dado que esta investigación se basó en un *snapshot* estático, una mejora clave consistiría en evaluar la evolución temporal del PageRank en PyPI a lo largo del tiempo, tomando datos mes a mes. Esto

permitiría predecir el ascenso de dependencias críticas antes de su consolidación global.

- **Ponderación de riesgo (Pesos en Aristas):** Modificar la matriz de transición para incorporar factores ponderados, tales como métricas de salud del proyecto (frecuencia de *commits*, número de mantenedores o "bus factor"), de manera que el *random walk* asigne mayor probabilidad a caminos más vulnerables.

Referencias

- [1] M. Zimmermann, C.-A. Staicu, C. Tenny y M. Pradel, «Small World with High Risks: A Study of Security Threats in the npm Ecosystem,» en *Proceedings of the 28th USENIX Security Symposium*, USENIX Association, 2019, págs. 995-1010.
- [2] A. Decan, T. Mens y P. Grosjean, «On the impact of security vulnerabilities in the npm package dependency network,» en *Proceedings of the 16th International Conference on Mining Software Repositories (MSR)*, IEEE, 2019, págs. 181-191. DOI: 10.1109/MSR.2019.00026
- [3] National Institute of Standards and Technology (NIST), *CVE-2021-44228 Detail*, <https://nvd.nist.gov/vuln/detail/CVE-2021-44228>, Log4Shell vulnerability, dic. de 2021.
- [4] R. Kikas, G. Gousios, M. Dumas y D. Pfahl, «Structure and evolution of package dependency networks,» en *Proceedings of the 14th International Conference on Mining Software Repositories (MSR)*, IEEE, 2017, págs. 102-112. DOI: 10.1109/MSR.2017.55
- [5] Tidelif, *Libraries.io Open Source Repository and Dependency Metadata*, Dataset de dependencias de paquetes PyPI, npm y otros ecosistemas, Zenodo, 2020. DOI: 10.5281/zenodo.3626071 dirección: <https://doi.org/10.5281/zenodo.3626071>
- [6] S. Brin y L. Page, «The anatomy of a large-scale hypertextual Web search engine,» en *Proceedings of the 7th International Conference on World Wide Web*, Elsevier, 1998, págs. 107-117. DOI: 10.1016/S0169-7552(98)00110-X
- [7] V. D. Blondel, J.-L. Guillaume, R. Lambiotte y E. Lefebvre, «Fast unfolding of communities in large networks,» *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, n.º 10, P10008, 2008. DOI: 10.1088/1742-5468/2008/10/P10008